

DK WAVE TECHNOLOGY

Azure Administrator Hands-On Lab

RAPPORT DE PHASE 5

Automatisation & Optimisation

Champ	Valeur
Auteur	Lancelot DOSSOU — Administrateur Systèmes & Réseaux Cloud
Date	26 Avril 2026
Projet	DK WAVE TECHNOLOGY — Azure Administrator Lab
Subscription	Azure Subscription 1 — ID : 1e1a51af-54c1-4759-9d61-c789627bcb06
Région principale	West Europe (westeurope)
Phases précédentes	Phase 1 à Phase 4 complétées
Statut Phase 5	Complétée avec adaptations terrain

Ce rapport documente l'intégralité des travaux réalisés, les choix techniques effectués, les outils utilisés et leur justification, ainsi que les difficultés rencontrées et les solutions appliquées.

1. Contexte du projet

1.1 Présentation de DK WAVE TECHNOLOGY

DK WAVE TECHNOLOGY est une entreprise éditrice de logiciels SaaS (Software as a Service) comptant 20 collaborateurs en télétravail. L'entreprise ne dispose d'aucun datacenter physique — l'intégralité de son infrastructure informatique est hébergée sur Microsoft Azure. Cette architecture 100% cloud impose une rigueur opérationnelle élevée : chaque service doit être disponible, sécurisé et maîtrisé financièrement.

1.2 Infrastructure existante à l'entrée de la Phase 5

Les quatre premières phases du laboratoire ont permis de construire une infrastructure Azure complète :

Phase	Thème	Réalisations principales
Phase 1	Fondations	Tenant Entra ID, groupes de ressources, utilisateurs & groupes, RBAC, MFA, Accès Conditionnel, Azure Policy — Tags obligatoires
Phase 2	Infrastructure	VNet 10.0.0.0/16, 6 sous-réseaux segmentés, NSG (web/app/db), Azure Bastion, VMs vm-app01 (Ubuntu) et vm-web02 (Windows Server 2025), Availability Set, Load Balancer
Phase 3	Sécurité	Microsoft Defender for Cloud, Key Vault kv-dkwave-secure avec Private Endpoint, Azure Firewall fw-dkwave-prod, WAF en mode Prevention, Resource Locks
Phase 4	Monitoring	Log Analytics Workspace law-dkwave-prod, Azure Monitor Agent, Data Collection Rule, alertes CPU > 80%, Action Group ag-it-ops, dashboards KQL

1.3 Problème identifié — Pourquoi cette phase est indispensable

Malgré une infrastructure fonctionnelle et sécurisée, l'ensemble des ressources avait été déployé manuellement. Cette approche présente trois risques majeurs en contexte professionnel :

Risque	Impact
ClickOps = Erreurs	Un déploiement manuel n'est jamais identique à 100%. Chaque intervention humaine introduit un risque d'erreur de configuration non reproductible et non auditable.
Dérive de configuration	Sans code de référence, les environnements Prod et NonProd divergent progressivement au fil des modifications manuelles. Après 6 mois, ils ne sont plus identiques.
Absence de traçabilité	Sans versioning Git, impossible de savoir qui a créé quoi, quand et pourquoi. En cas d'incident, la reconstruction est longue, incertaine et coûteuse.

Principe fondateur de la Phase 5

"Si ce n'est pas dans le code, ca n'existe pas." Cette phase transforme une infrastructure construite a la main en un environnement pilote par le code, automatise et financièrement maitrise.

2. Objectifs de la Phase 5

#	Objectif	Résultat attendu
1	Infrastructure as Code avec Bicep	Templates déployables en une commande, versionnables dans Git
2	Paramétrage multi-environnements	Un seul template, plusieurs environnements (Prod/NonProd)
3	Automatisation du déploiement VM	Plus aucun déploiement manuel de machine virtuelle
4	Patching automatique via Update Manager	Mises à jour de sécurité appliquées sans intervention humaine
5	Tags comme leviers de gouvernance	100% des ressources taguées, gouvernance active
6	Cost Management avec alertes	Budget mensuel surveillé avec notifications proactives
7	Right-sizing des ressources	Identification et documentation des économies possibles

3. Outils et services utilisés — Justification

3.1 Azure Bicep — Infrastructure as Code

Attribut	Détail
Qu'est-ce que c'est ?	Bicep est le langage d'Infrastructure as Code natif de Microsoft Azure. Il compile en templates ARM (Azure Resource Manager) et décrit l'état désiré d'une infrastructure.
Pourquoi Bicep et pas Terraform ?	Bicep est le choix optimal pour un environnement 100% Azure : syntaxe plus concise que les templates ARM bruts, intégration native sans provider à configurer, support complet de toutes les ressources Azure dès le premier jour.
Propriété clé : Idempotence	Un même template peut être déployé 100 fois : si la ressource existe déjà et est conforme, Azure ne fait rien. Si elle a été modifiée, Azure la remet à l'état désiré. Cela élimine les erreurs de double création.
A quoi ca sert ensuite ?	Versioning Git des templates, revue de code des changements d'infrastructure, déploiements automatisés via CI/CD (Azure DevOps, GitHub Actions), reconstruction en cas de disaster recovery.

3.2 Azure Cloud Shell — Environnement d'exécution

Attribut	Détail
Qu'est-ce que c'est ?	Terminal Linux hébergé dans Azure, accessible depuis le navigateur. Pré-configuré avec Azure CLI, Bicep, Python, et un stockage persistant Azure Files.
Pourquoi l'utiliser ?	Authentification automatique avec le compte Azure connecté, éliminant les risques de déploiement dans la mauvaise subscription. Environnement standardisé, indépendant de la machine locale.
A quoi ca sert ensuite ?	Exécution de scripts d'automatisation, déploiements d'urgence depuis n'importe quel navigateur sans VPN, environnement de test isolé.

3.3 Azure CLI et API REST Azure

Attribut	Détail
Azure CLI	Interface en ligne de commande pour gérer les ressources Azure. Utilisée pour les déploiements Bicep, la gestion des VMs, les tags et les métriques.
API REST Azure (az rest)	Accès direct à l'API ARM d'Azure, contournant les limitations des extensions CLI en preview. Utilisé pour configurer patchMode et bypassPlatformSafetyChecksOnUserSchedule sur les VMs.
Pourquoi les deux ?	La CLI couvre 90% des besoins avec une syntaxe conviviale. L'API REST prend le relais pour les fonctionnalités avancées non encore exposées dans les extensions CLI stables.
A quoi ca sert ensuite ?	Scripting d'automatisation, pipelines CI/CD, intégration dans des outils de gestion de configuration (Ansible, etc.).

3.4 Azure Update Manager

Attribut	Détail
Qu'est-ce que c'est ?	Service Azure centralisé pour gérer les mises à jour de systèmes d'exploitation sur les VMs Azure (Windows et Linux). Successeur de la solution Update Management dans Azure Automation.
Pourquoi l'utiliser ?	Les systèmes non patchés sont le principal vecteur d'attaque par ransomware. Le patching manuel est systématiquement différé sous charge opérationnelle. L'automatisation garantit une conformité continue.
Modes de patch configurés	AutomaticByPlatform : Azure orchestre le patching selon la fenêtre de maintenance définie. Les redémarrages sont gérés automatiquement si nécessaires.
A quoi ca sert ensuite ?	Conformité RGPD et audits de sécurité (preuve de patching régulier), réduction de la surface d'attaque, reporting centralisé des mises à jour appliquées.

3.5 Azure Cost Management

Attribut	Détail
Qu'est-ce que c'est ?	Service Azure de surveillance et d'analyse des coûts cloud. Permet de définir des budgets, configurer des alertes et analyser les dépenses par ressource, tag ou service.
Pourquoi l'utiliser ?	En cloud, les ressources sont facturées même inactives. Un administrateur Azure est responsable des coûts, pas uniquement de la technique. Un dépassement non détecté peut avoir des impacts financiers majeurs.
Important a retenir	Le budget ne coupe PAS les ressources. Il envoie uniquement des alertes. L'action reste humaine.
A quoi ca sert ensuite ?	Tableaux de bord management, imputation des coûts par département (via tags CostCenter), optimisation continue, préparation des budgets annuels IT.

3.6 Azure Monitor — Métriques pour Right-sizing

Attribut	Détail
Qu'est-ce que c'est ?	Service de monitoring Azure collectant des métriques en temps réel sur toutes les ressources (CPU, mémoire, réseau, disque). Déjà configuré en Phase 4.
Utilisation en Phase 5	Requête des métriques CPU historiques de vm-app01 sur 7 jours pour identifier un surdimensionnement.
A quoi ca sert ensuite ?	Right-sizing continu (réduction des coûts), capacity planning, détection des goulots d'étranglement, optimisation des performances applicatives.

4. Réalisations détaillées

4.1 Étape 5.2 — Préparation de l'environnement IaC

Ce qui a été fait

Vérification de la subscription active et initialisation du répertoire de projet IaC dans Cloud Shell.

```
az account show
# Résultat : Subscription 1e1a51af-54c1-4759-9d61-c789627bcb06 - Azure
subscription 1

mkdir ~/dkwave-iac && cd ~/dkwave-iac
```

Pourquoi c'est important

La vérification de la subscription active avant tout déploiement est une discipline fondamentale. Déployer dans la mauvaise subscription est l'une des erreurs les plus coûteuses et les plus difficiles à corriger en production. Azure Cloud Shell authentifie automatiquement l'utilisateur connecté au portail, garantissant un contexte de déploiement cohérent et traçable.

4.2 Étape 5.3 — Premier template Bicep

Ce qui a été fait

Création du fichier main.bicep décrivant un compte de stockage Azure avec la fonction uniqueString() pour garantir un nom globalement unique, et les 4 tags obligatoires. Déploiement réussi dans rg-dkwave-prod.

```
# Contenu de main.bicep
param location string = 'westeurope'
param storageAccountName string = 'stdkwave${uniqueString(resourceGroup().id)}'
param environment string = 'Prod'

resource storage 'Microsoft.Storage/storageAccounts@2022-09-01' = {
  name: storageAccountName
  location: location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
  tags: {
    Environment: environment
    Company: 'DK-WAVE'
    Owner: 'IT'
    CostCenter: 'IT-OPS'
  }
}
```

Résultat	Valeur
Ressource créée	stdkwavecbg3cqgonj4p4 (Microsoft.Storage/storageAccounts)
Durée 1er déploiement	23.5 secondes

Durée 2ème déploiement	1.3 secondes (idempotence — aucun changement détecté)
provisioningState	Succeeded

Pourquoi l'idempotence est fondamentale

La durée de 1.3 secondes au second déploiement illustre l'idempotence de Bicep : Azure compare l'état désiré (le code) avec l'état réel (la ressource existante) et ne fait rien si les deux correspondent. Cette propriété permet de rejouer les déploiements sans risque — par exemple après un incident pour s'assurer que l'infrastructure est bien dans l'état attendu.

4.3 Étape 5.4 — Paramétrage multi-environnements

Ce qui a été fait

Création de deux fichiers de paramètres distincts et déploiement du même template dans deux environnements différents.

```
# Déploiement Prod
az deployment group create \
  --resource-group rg-dkwave-prod \
  --template-file main.bicep \
  --parameters prod.parameters.json

# Déploiement NonProd
az deployment group create \
  --resource-group rg-dkwave-nonprod \
  --template-file main.bicep \
  --parameters nonprod.parameters.json
```

Environnement	Ressource créée	Tag Environment	Durée
rg-dkwave-prod	stdkwavecbg3cqgonj4p4	Prod	1.3s (idempotent)
rg-dkwave-nonprod	stdkwaveqv57mpfiqz6ng	NonProd	22s (création)

Pourquoi c'est important

Un seul fichier main.bicep sert les deux environnements. La seule différence est la valeur du paramètre environment dans chaque fichier de paramètres. Toute correction ou évolution du template s'applique uniformément à Prod et NonProd, éliminant les risques de dérive de configuration entre environnements.

4.4 Étape 5.5 — Automatisation du déploiement VM

Ce qui a été fait

Création du template vm.bicep décrivant une machine virtuelle Ubuntu complète avec sa NIC, ses tags, sa configuration réseau privée et la protection du mot de passe administrateur.

```
# Points techniques clés du template vm.bicep

@secure() # Masque le mot de passe dans les logs
param adminPassword string

resource nic 'Microsoft.Network/networkInterfaces@2022-07-01' = {
  # NIC sans IP publique — accès via Bastion uniquement
  properties: {
    ipConfigurations: [{
      properties: {
        subnet: {
          id: resourceId('rg-dkwave-shared',
            'Microsoft.Network/virtualNetworks/subnets',
            'vnet-dkwave-prod', 'subnet-app')
        }
        privateIPAllocationMethod: 'Dynamic'
      }
    }]
  }
}

resource vm 'Microsoft.Compute/virtualMachines@2022-08-01' = {
  tags: {
    AutoShutdown: 'False' # Tag pour automatisation extinction nuit
  }
  properties: {
    networkProfile: {
      networkInterfaces: [{ id: nic.id }] # Dépendance implicite NIC -> VM
    }
  }
}
```

Points techniques notables

- `@secure()` : Bicep ne stocke jamais ce paramètre en clair dans les logs de déploiement ni dans l'historique de l'API ARM. C'est la pratique obligatoire pour tous les secrets.
- Dépendance implicite via `nic.id` : Bicep analyse les références entre ressources et crée automatiquement l'ordre de déploiement correct (NIC d'abord, VM ensuite). Pas besoin de `dependsOn` explicite.
- Aucune IP publique : la VM est accessible uniquement via Azure Bastion sur le réseau privé. La surface d'attaque est réduite au minimum.
- Tag `AutoShutdown` : prévu pour des scripts d'automatisation qui pourront cibler et éteindre automatiquement les VMs NonProd la nuit, générant des économies substantielles.

4.5 Étape 5.6 — Azure Update Manager

Ce qui a été fait

Configuration de la politique de maintenance patch-config-prod et attachement des deux VMs de production.

Paramètre	Valeur configurée
Nom	patch-config-prod

Planification	1er du mois, 22h00 (UTC+1 Paris/Bruxelles)
Fenêtre de maintenance	3 heures
Paramètre de redémarrage	Redémarrer si nécessaire
Classifications Windows	Critical Updates + Security Updates
Classifications Linux	Critical + Security
VMs attachées	vm-app01 (Linux Ubuntu 24.04) + vm-web02 (Windows Server 2025)
Mode de patch	AutomaticByPlatform sur les deux VMs

Configuration via API REST — Pourquoi

Les prérequis techniques d'Azure Update Manager (patchMode=AutomaticByPlatform et bypassPlatformSafetyChecksOnUserSchedule=true) n'étaient pas accessibles via la CLI stable. L'API REST Azure a été utilisée directement, ce qui illustre une compétence avancée d'administrateur : savoir contourner les limitations des outils CLI en accédant directement à l'API ARM.

```
az rest --method patch \  
  --url "https://management.azure.com/subscriptions/.../virtualMachines/vm-  
app01?api-version=2023-03-01" \  
  --body '{"properties":{"osProfile":{"linuxConfiguration":{"patchSettings":{"  
    "patchMode":"AutomaticByPlatform",  
    "automaticByPlatformSettings":{"  
      "bypassPlatformSafetyChecksOnUserSchedule":true  
    }}}}}}'  
  
# Résultat : provisioningState: Updating -> Succeeded
```

4.6 Étape 5.7 — Tags comme leviers de gouvernance

Ce qui a été fait

Audit complet des tags sur rg-dkwave-prod, identification des ressources non conformes et correction systématique.

```
# Audit : identification des ressources sans tags  
az resource list \  
  --resource-group rg-dkwave-prod \  
  --query "[?tags==null].{Nom:name, Type:type}" \  
  --output table  
  
# Résultat : 6 ressources non conformes identifiées  
# NICs, Disques OS, IP publique AppGW, Politique WAF
```

Ressource	Type	Statut
vm-app01871_z1	Microsoft.Network/networkInterfaces	Corrigé
vm-web02284_z1	Microsoft.Network/networkInterfaces	Corrigé

vm-app01_OsDisk_1_...	Microsoft.Compute/disks	Corrigé
vm-web02_OsDisk_1_...	Microsoft.Compute/disks	Corrigé
pip-appgw	Microsoft.Network/publicIPAddresses	Corrigé
waf-dkwave	Microsoft.Network/applicationGatewayWebApplicationFirewallPolicies	Corrigé via az rest

Les tags comme déclencheurs d'automatisation

Au-delà de l'organisation visuelle, les tags sont utilisés comme logique de ciblage pour des scripts d'automatisation. Le tag AutoShutdown=True permet par exemple d'éteindre automatiquement toutes les VMs NonProd chaque soir sans intervention manuelle, générant des économies significatives sur la facture mensuelle Azure.

4.7 Étape 5.8 — Cost Management & Budget

Ce qui a été fait

Création du budget mensuel dkwave-budget-prod-2026 avec deux seuils d'alerte email automatiques.

Paramètre	Valeur
Nom du budget	dkwave-budget-prod-2026
Montant mensuel	150 USD
Période	Mensuelle (reset automatique chaque mois)
Alerte 1 (avertissement)	80% du budget = 120 USD — email automatique
Alerte 2 (critique)	100% du budget = 150 USD — email automatique
Destinataire	dossam2006@gmail.com
Date d'expiration	31 Mars 2028

Validation en conditions réelles

Le système d'alerte a été valide en production réelle : le 27 Avril 2026 à 00h19 UTC, une alerte automatique a été reçue confirmant que les dépenses du mois atteignaient 150.77 USD (dépassement du seuil de 150 USD). Le système fonctionne comme prévu.

4.8 Étape 5.9 — Right-sizing des ressources

Ce qui a été fait

Analyse des métriques CPU historiques de vm-app01 via Azure Monitor pour identifier un potentiel surdimensionnement.

```
az monitor metrics list \
  --resource /subscriptions/.../virtualMachines/vm-app01 \
  --metric "Percentage CPU" \
  --interval PT1H \
  --aggregation Average \
  --query "value[0].timeseries[0].data[-5:].average" \
  --output table
```

Période	CPU moyen	Seuil normal	Analyse
H-4	0.86%	< 20% = sous-utilisé	Sous-utilisation critique
H-3	0.49%	< 20% = sous-utilisé	Sous-utilisation critique
H-2	0.49%	< 20% = sous-utilisé	Sous-utilisation critique
H-1	0.49%	< 20% = sous-utilisé	Sous-utilisation critique
Moyenne	< 1%	Objectif : 40-70%	Recommandation : Downsizing

Recommandation documentée

VM actuelle : Standard_D2ads_v6 (2 vCPUs) → VM recommandée : Standard_D1ads_v6 (1 vCPU). Économie estimée : environ 35 USD/mois soit ~420 USD/an. En production, cette recommandation serait soumise à validation lors de la prochaine fenêtre de maintenance mensuelle.

5. Difficultés rencontrées et solutions appliquées

5.1 Quota de vCPUs épuisé — Étape 5.5

Erreur rencontrée

QuotaExceeded: Operation could not be completed as it results in exceeding approved Total Regional Cores quota. Current Limit: 4, Current Usage: 4, Additional Required: 2

Analyse

La subscription Azure Student/gratuite dispose d'un quota limité à 4 vCPUs en région West Europe. Les VMs existantes vm-app01 et vm-web02 (Standard_D2ads_v6, 2 vCPUs chacune) consomment l'intégralité du quota. Plusieurs tentatives d'adaptation ont été menées :

- Réduction à Standard_B1s (1 vCPU) : quota non libéré en temps réel après désallocation de vm-app01
- Déploiement en North Europe : Standard_B1s et Standard_B2s indisponibles dans cette région au moment du déploiement
- Standard_D2ads_v6 en North Europe : même résultat, quota West Europe non partagé avec North Europe

Décision prise

Le template vm.bicep est syntaxiquement correct et validé par l'API Azure. L'obstacle est une contrainte de quota subscription, non une erreur technique. En entreprise, la procédure standard est une demande d'augmentation de quota via le portail Azure Support (délai : 24-48h). Cette situation illustre un scénario réel fréquent en environnement professionnel.

5.2 Extensions CLI Azure en version preview — Étapes 5.6 et 5.8

Erreurs rencontrées

az maintenance configuration create : unrecognized arguments --reboot-setting, Operation returned an invalid status 'Bad Request'
az consumption budget create : Invalid budget configuration, please use filter interface with 2019-05-01-preview version

Analyse

Les extensions Microsoft.Maintenance et az consumption sont en version preview dans Azure CLI. Ces extensions présentent des comportements non documentés : arguments manquants, incompatibilités de version API, erreurs Bad Request sans message détaillé.

Solution appliquée

Identification rapide de l'instabilité et basculement vers des alternatives stables selon le principe du moindre effort :

- Configuration de maintenance : portail Azure (interface stable, guidée, sans bug de version)
- Budget : portail Azure Cost Management (interface dédiée et complète)

- Paramètres VM avancés : az rest avec l'API REST directe (contourne toutes les limitations CLI)

Enseignement

Un administrateur Azure expérimenté maîtrise plusieurs approches pour atteindre un objectif (CLI, portail, API REST, PowerShell, Bicep) et sait identifier rapidement quand une approche est contreproductive. La capacité à choisir pragmatiquement l'outil adapté est une compétence différenciante.

5.3 Prérequis complexes d'Azure Update Manager — Étape 5.6

Erreur rencontrée

The prerequisites to patch your machine were not met. Please set the patchMode to AutomaticByPlatform and bypassPlatformChecksOnUserSchedule as true.

Complexité additionnelle — vm-web02

vm-web02 utilisait le Hotpatching Windows (fonctionnalité Windows Server 2025 permettant d'appliquer des patches sans redémarrage), incompatible avec `bypassPlatformSafetyChecksOnUserSchedule=true`. L'activation forcée retournait : Hotpatch updates require platform safety checks. The `bypassPlatformSafetyChecksOnUserSchedule` flag must be set to false.

Solution en deux étapes

- Désactivation du Hotpatching sur vm-web02 via API REST (`enableHotpatching: false`)
- Configuration de `bypassPlatformSafetyChecksOnUserSchedule=true` sur les deux VMs
- Résultat : les deux VMs attachées avec succès à patch-config-prod

Enseignement

La compréhension des dépendances entre fonctionnalités Azure (Hotpatching est incompatible avec le mode User Schedule) est essentielle. Ces incompatibilités ne sont pas toujours documentées clairement et nécessitent une lecture attentive des messages d'erreur et une exploration méthodique des paramètres disponibles.

5.4 Timeouts réseau et comportement asynchrone Azure — Étape 5.6

Erreur rencontrée

La demande a expiré. Informations de diagnostic : horodatage 20260426T212123Z, ID de suivi dc045697-d006-4e9a-a64e-24d779d6b1ec. Echec de 1 affectation(s).

Analyse

Azure est un système distribué et asynchrone. Certaines opérations longues (attachement de VMs à une configuration de maintenance) dépassent le timeout du client HTTP du portail Azure. Le message d'erreur côté client ne reflète pas l'état réel côté serveur.

Solution appliquée

Vérification de l'état réel de la ressource dans le portail après le timeout : les deux VMs apparaissaient bien dans l'onglet Ressources de patch-config-prod, confirmant que l'opération avait été traitée avec succès en arrière-plan par Azure.

Enseignement

La bonne pratique est toujours de vérifier l'état réel de la ressource (portail ou `az resource show`) plutôt que de se fier uniquement au message affiché dans le client. Un timeout côté client ne signifie pas un échec côté serveur.

5.5 Gestion des coûts — Alerte budget en conditions réelles

Alerte reçue

Objet : Budget dkwave-budget-prod-2026 has exceeded its threshold value. Total evaluated: 150.77 USD. Threshold: 150.00 USD. Alert generation time: April 27 2026 0:19 UTC.

Analyse et action

Le dépassement de budget a été causé principalement par Azure Firewall (~30 EUR/jour), Azure Bastion (~4 EUR/jour) et Application Gateway (~6 EUR/jour), qui tournaient en continu depuis leur création en Phase 2 et Phase 3. Ces ressources sont nécessaires pour le lab mais coûteuses à laisser allumées en permanence.

Actions correctives immédiatement appliquées

- Désallocation de vm-app01 et vm-web02 : 0 EUR/heure en compute (disques seuls facturés)
- Suppression d'Azure Firewall fw-dkwave-prod : économie de ~30 EUR/jour
- Suppression d'Azure Bastion bastion-dkwave : économie de ~4 EUR/jour
- Suppression d'Application Gateway appgw-dkwave : économie de ~6 EUR/jour
- Suppression de la politique WAF waf-dkwave (liée à l'AppGW)

Enseignement clé

En cloud, la responsabilité financière fait partie intégrante du rôle d'administrateur. Les ressources Azure sont facturées même inactives. La mise en place du budget en Phase 5 a permis de détecter et corriger une dérive de coûts avant qu'elle ne devienne incontrôlable. Les ressources supprimées seront recréées via Bicep en Phase 6 — c'est précisément la valeur de l'IaC mis en place dans cette phase.

6. Tableau de synthèse des réalisations

Étape	Objectif	Réalisation	Statut
5.2	Environnement IaC	Cloud Shell configuré, subscription vérifiée, dkwave-iac/ créé	Validée
5.3	Premier Bicep	main.bicep déployé, stdkwavecbg3cqgonj4p4 créé, idempotence démontrée	Validée
5.4	Multi-environnements	Déploiements Prod et NonProd avec fichiers de paramètres distincts	Validée
5.5	VM via IaC	vm.bicep complet et validé, déploiement bloqué par quota subscription	Template prêt
5.6	Update Manager	patch-config-prod créé, vm-app01 et vm-web02 attachées, planification mensuelle active	Validée
5.7	Tags gouvernance	Audit complet, 6 ressources corrigées, 100% des ressources taguées	Validée
5.8	Budget & alertes	Budget 150 USD/mois, alertes 80% et 100%, validé en conditions réelles	Validée
5.9	Right-sizing	CPU < 1% sur vm-app01, recommandation Standard_D1ads_v6 documentée	Validée

7. Compétences démontrées

Domaine	Compétences
Infrastructure as Code	Templates Bicep avec paramètres, fonctions uniqueString() et resourceId(), déploiements multi-environnements, sécurisation des secrets avec @secure(), idempotence
Automatisation opérationnelle	Configuration Azure Update Manager, utilisation de l'API REST Azure (az rest), audit et normalisation des tags, création de budgets et alertes Cost Management
Analyse et optimisation	Analyse de métriques Azure Monitor, identification du surdimensionnement, recommandation de right-sizing documentée et chiffrée

Résolution de problèmes	Diagnostic des erreurs de quota, gestion des incompatibilités Hotpatching, adaptation aux extensions CLI instables, interprétation des comportements asynchrones Azure
Bonnes pratiques entreprise	Documentation des recommandations non appliquées, principe IaC (code = source de vérité), traçabilité des décisions, gestion proactive des coûts cloud

8. Conclusion

La Phase 5 représente une transition fondamentale dans la maturité de l'infrastructure DK WAVE TECHNOLOGY : le passage d'une infrastructure construite manuellement à un environnement piloté par le code, automatisé et financièrement maîtrisé.

Les apports concrets et mesurables de cette phase sont les suivants : une base IaC opérationnelle avec Bicep prouvée par deux déploiements réussis et versionnables dans Git, un système de patching automatique mensuel couvrant les deux VMs de production sans intervention humaine, une gouvernance par tags normalisée sur 100% des ressources contrôlables de rg-dkwave-prod, un contrôle budgétaire actif avec alertes proactives validé en conditions réelles, et une recommandation de right-sizing documentée représentant une économie annuelle estimée de 420 USD.

Les difficultés rencontrées — quota insuffisant, extensions CLI instables, prérequis complexes d'Update Manager, timeouts réseau, alerte de dépassement de budget — reflètent fidèlement les contraintes d'un environnement Azure professionnel réel. La capacité à adapter la stratégie en cours d'exécution, à documenter honnêtement les obstacles et à choisir pragmatiquement les outils les plus adaptés à chaque situation démontre une maturité opérationnelle alignée avec les exigences du marché de l'emploi en administration cloud.

Message aux recruteurs

Ce rapport documente un travail pratique réalisé en conditions réelles sur une subscription Azure payante. Chaque commande a été exécutée, chaque erreur rencontrée, chaque solution testée. L'objectif n'est pas de montrer un parcours sans difficulté, mais de démontrer la capacité à résoudre des problèmes réels de manière méthodique et professionnelle.

DK WAVE TECHNOLOGY — Phase 5 complétée

Lancelot DOSSOU — Administrateur Systèmes & Réseaux Cloud
26 Avril 2026